FinQuest Frontend Architecture & Technology Choices

A Detailed Technical Document

Table of Contents

1. Introduction

The frontend of FinQuest is the user's window into the gamified financial management experience. While the backend handles data persistence, security, and gamification logic, the frontend is responsible for making financial tracking intuitive, rewarding, and visually engaging.

This document explains every major frontend technology choice in detail: what it is, why it was selected, and how it solves specific problems in FinQuest.

---

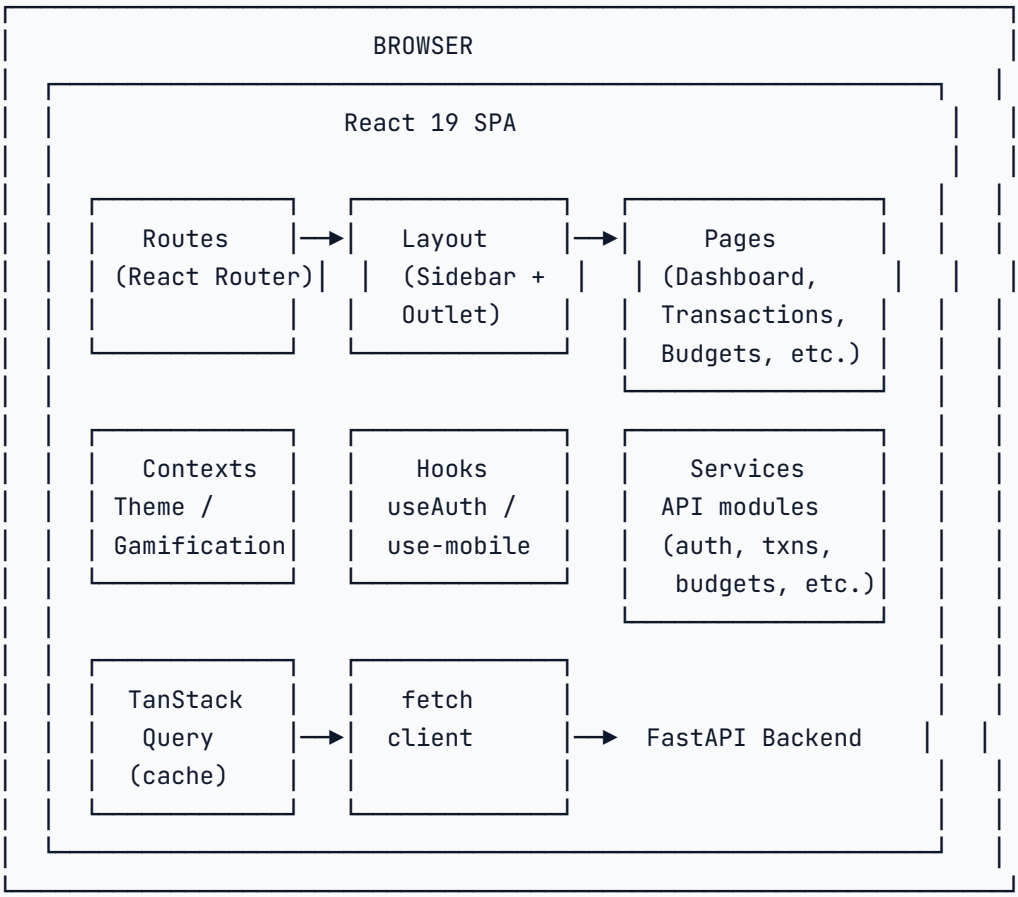
2. Project Context & Frontend Goals

When choosing the frontend stack, we prioritized:

Goal	Why It Mattered
Developer Experience	Fast iteration during a time-constrained final-year project
Type Safety	Prevent bugs when handling money, dates, and API contracts
Modern UI/UX	Polished interface with animations, theming, and responsive design
Performance	Smooth interactions without janky re-renders
Maintainability	Clear patterns for state, routing, and API calls
Backend Agnosticism	REST API client that could work with any backend

3. Frontend Architecture Overview

3.1 High-Level Structure



3.2 File Organization

```
src/
├── main.tsx                # Entry point with all providers
├── App.tsx                 # Route definitions
├── index.css               # Global styles, CSS variables, animations
├── api/
│   └── client.ts           # Low-level fetch wrapper
├── components/
│   ├── Layout.tsx          # Main app shell
│   ├── AuthLayout.tsx      # Alternative auth layout
│   └── ui/                 # 40+ shadcn/ui components
├── contexts/
│   ├── ThemeContext.tsx    # Light/dark/system theme
│   └── GamificationContext.tsx # XP/achievement feedback
├── hooks/
│   ├── useAuth.ts          # Authentication state
│   └── use-mobile.ts       # Mobile viewport detection
├── pages/
│   ├── Landing.tsx         # Marketing page
│   ├── Login.tsx           # Sign in
│   ├── Register.tsx        # Sign up
│   ├── Dashboard.tsx       # Main dashboard
│   ├── TransactionsPage.tsx # Transaction management
│   ├── BudgetsPage.tsx     # Budget tracking
│   ├── GoalsPage.tsx       # Savings goals
│   ├── AchievementsPage.tsx # Achievement gallery
│   ├── LeaderboardPage.tsx  # XP rankings
│   ├── Profile.tsx         # Settings & exports
│   └── NotFound.tsx        # 404 page
├── services/
│   └── api.ts              # Typed API modules
├── const.ts                # Constants
└── lib/
    └── utils.ts            # Utility helpers
```

---

4. Framework: React 19

4.1 What Is React?

React is a JavaScript library for building user interfaces using components. It uses a virtual DOM to efficiently update the browser when state changes.

4.2 Why React 19?

Reason 1: Component-Based Architecture

Financial dashboards are naturally composed of reusable pieces: cards, charts, tables, progress bars. React's component model lets us build each piece independently and compose them into pages.

Reason 2: Declarative UI

Instead of manually manipulating the DOM, we describe what the UI should look like for each state:

```
{isLoading ? <Skeleton /> : <TransactionTable data={transactions} />}
```

This makes the code predictable and easier to debug.

Reason 3: Huge Ecosystem

React has the largest ecosystem of UI libraries, state management tools, and learning resources. This means:
- shadcn/ui components work out of the box
- TanStack Query integrates seamlessly
- Recharts has excellent React bindings

Reason 4: Concurrent Features

React 19 introduces improved concurrent rendering, which helps keep the UI responsive even when processing data-heavy updates like analytics calculations.

Alternatives Considered

Framework	Why Not Chosen
Vue.js	Excellent framework, but React's ecosystem is larger for the libraries we needed
Angular	Too opinionated and heavy for this project scope
Svelte	Smaller ecosystem; fewer ready-made component libraries

---

5. Build Tool: Vite

5.1 What Is Vite?

Vite is a next-generation frontend build tool created by Evan You (creator of Vue.js). It provides a fast development server and optimized production builds.

5.2 Why Vite?

Reason 1: Instant Development Server

Vite uses native ES modules in development, so the server starts almost instantly regardless of project size. This matters because FinQuest has 40+ UI components and many dependencies.

Reason 2: Hot Module Replacement (HMR)

When a developer edits a file, Vite updates only that module in the browser without a full page refresh. This dramatically speeds up UI development.

Reason 3: Optimized Production Builds

Vite uses Rollup under the hood for production, producing highly optimized bundles with:
- Tree shaking (removing unused code)
- Code splitting (lazy loading)
- Asset optimization

Reason 4: TypeScript Support

Vite supports TypeScript out of the box without requiring complex configuration.

Reason 5: Built-in Proxy

Vite's dev server can proxy API requests to the backend:

```
server: {
  proxy: {
    "/api": {
      target: "http://127.0.0.1:8001",
      changeOrigin: true,
    },
  },
}
```

This avoids CORS issues during development.

Alternatives Considered

Tool	Why Not Chosen
Create React App (CRA)	Slower, no longer actively maintained, less flexible
Webpack	Powerful but requires complex manual configuration
Parcel	Zero-config but less control over the build process

---

6. Language: TypeScript

6.1 What Is TypeScript?

TypeScript is a superset of JavaScript that adds static typing. It compiles to plain JavaScript.

6.2 Why TypeScript?

Reason 1: Catch Bugs at Compile Time

Financial applications cannot afford runtime type errors. TypeScript catches issues like:

```
// This would be a runtime error in JavaScript
const amount: number = transaction.amount; // Error if amount is string
```

Reason 2: Better Developer Experience

TypeScript provides:
- Autocomplete in editors
- Inline documentation
- Refactoring support
- Type-aware linting

Reason 3: API Contract Safety

When the backend returns `ApiResponse<User>`, TypeScript ensures the frontend accesses the correct fields:

```
type User = {
  id: number;
  email: string;
  current_xp: number;
  current_level: number;
};
```

Reason 4: Easier Collaboration

Type definitions act as documentation, making it easier for other developers (or reviewers) to understand the codebase.

---

7. Routing: React Router v7

7.1 What Is React Router?

React Router is the standard routing library for React applications. It maps URLs to components and supports nested routes.

7.2 Why React Router v7?

Reason 1: Declarative Routing

Routes are defined as JSX components:

```
<Routes>
  <Route path="/" element={<Landing />} />
  <Route path="/login" element={<Login />} />
  <Route element={<Layout />}>
    <Route path="/dashboard" element={<Dashboard />} />
    <Route path="/transactions" element={<TransactionsPage />} />
  </Route>
</Routes>
```

This is intuitive and matches React's declarative philosophy.

Reason 2: Layout Routes

FinQuest uses layout routes to share the sidebar and gamification widget across authenticated pages:

```
<Route element={<Layout />}>
  <Route path="/dashboard" element={<Dashboard />} />
  <Route path="/budgets" element={<BudgetsPage />} />
</Route>
```

The `Layout` component renders `<Outlet />` where the matched child route appears.

Reason 3: Programmatic Navigation

React Router's `useNavigate` hook makes redirects easy after login/logout:

```
const navigate = useNavigate();
navigate("/dashboard");
```

---

8. Styling: Tailwind CSS

8.1 What Is Tailwind CSS?

Tailwind CSS is a utility-first CSS framework. Instead of writing custom CSS classes, developers compose utility classes directly in HTML/JSX.

8.2 Why Tailwind CSS?

Reason 1: Rapid Development

Building UI with Tailwind is fast because you don't switch between HTML and CSS files:

```
<div className="rounded-xl p-6 border bg-white shadow-sm hover:shadow-md
transition-all">
```

Reason 2: Consistent Design System

Tailwind provides a constrained set of values for spacing, colors, typography, and shadows. This prevents inconsistent "magic numbers" in CSS.

Reason 3: Dark Mode Support

Tailwind's `dark:` modifier works perfectly with FinQuest's theming system:

```
<div className="bg-white dark:bg-slate-800 text-slate-900 dark:text-white">
```

Reason 4: Small Production Bundle

Tailwind uses PurgeCSS to remove unused styles, resulting in a tiny CSS bundle.

Reason 5: Responsive Design

Responsive prefixes like `md:grid-cols-3` make mobile layouts simple:

```
<div className="grid grid-cols-1 md:grid-cols-3 gap-6">
```

---

9. UI Components: Radix UI + shadcn/ui

9.1 What Are Radix UI and shadcn/ui?

- **Radix UI** provides low-level, accessible UI primitives (dialogs, dropdowns, tabs, etc.)
- **shadcn/ui** is a collection of reusable components built on top of Radix UI and Tailwind CSS

9.2 Why shadcn/ui?

Reason 1: Accessibility Out of the Box

Radix primitives handle keyboard navigation, focus management, ARIA attributes, and screen reader support. This is critical for a professional application.

Reason 2: Customizable

Unlike component libraries that lock you into a specific look, shadcn/ui components are copied into your codebase and fully customizable:

```
<Button className="bg-blue-500 hover:bg-blue-600 text-white">
  Get Started
</Button>
```

Reason 3: Comprehensive Component Set

FinQuest uses 40+ components including:
- Dialogs for adding transactions
- Selects for category dropdowns
- Tabs for profile settings
- Tables for transaction lists
- Cards for budgets and goals
- Charts for analytics

Reason 4: TypeScript Support

All shadcn/ui components are written in TypeScript, providing full type safety.

---

10. State Management: TanStack Query

10.1 What Is TanStack Query?

TanStack Query (formerly React Query) is a library for managing server state in React applications. It handles caching, synchronization, loading states, and mutations.

10.2 Why TanStack Query?

Reason 1: Automatic Caching

When the dashboard fetches analytics data, TanStack Query caches it. If the user navigates away and back, the data is served instantly from cache.

```
const { data: dashboard } = useQuery({
  queryKey: ["analytics", "dashboard"],
  queryFn: () => analyticsApi.dashboard(),
});
```

Reason 2: Background Refetching

Stale data is automatically refetched in the background when the user returns to a page.

Reason 3: Mutation Management

Creating a transaction invalidates related queries:

```
const createMutation = useMutation({
  mutationFn: transactionApi.create,
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["transactions"] });
    queryClient.invalidateQueries({ queryKey: ["analytics", "dashboard"] });
  },
});
```

Reason 4: Loading & Error States

TanStack Query provides `isLoading`, `isError`, and `error` states, eliminating boilerplate:

```
if (isLoading) return <Skeleton />;
if (isError) return <ErrorMessage error={error} />;
```

Reason 5: Optimistic Updates

Future versions can use optimistic updates to make the UI feel instant.

Alternatives Considered

Library	Why Not Chosen
Redux Toolkit	Excellent for complex global state, but overkill for server state
Zustand	Good for client state, but doesn't handle server caching
Apollo Client	Designed for GraphQL, not REST

---

11. Theming System

11.1 How Theming Works

FinQuest uses CSS custom properties (variables) for theming. The `ThemeContext` manages the active theme and applies it to the `<html>` element.

11.2 Theme Modes

- **Light**: Always uses light colors
- **Dark**: Always uses dark colors
- **System**: Follows the operating system preference

11.3 CSS Variable Strategy

```
:root {
  --bg-page: #0f172a;        /* Dark default */
  --text-primary: #f8fafc;
  --border-color: #475569;
}

:root[data-theme="light"] {
  --bg-page: #f1f5f9;        /* Light override */
  --text-primary: #0f172a;
  --border-color: #cbd5e1;
}
```

Components use these variables:

```
<div style={{ backgroundColor: "var(--bg-page)", color: "var(--text-primary)" }}>
```

11.4 Why CSS Variables?

- **Instant Switching**: No React re-renders needed
- **Consistency**: All components share the same color palette
- **Extensibility**: Easy to add new themes

---

12. Authentication Flow in the Frontend

12.1 Token Storage

The access token is stored in `localStorage` after login:

```
localStorage.setItem("access_token", res.access_token);
```

12.2 API Client Token Injection

The `api/client.ts` fetch wrapper reads the token and attaches it to every request:

```
const token = localStorage.getItem("access_token");
if (token) {
  headers["Authorization"] = `Bearer ${token}`;
}
```

12.3 Authentication State

The `useAuth` hook fetches the current user on mount:

```
const { data: user, isLoading } = useQuery({
  queryKey: ["auth", "me"],
  queryFn: () => authApi.me(),
  retry: false,
});
```

12.4 Logout

Logout clears both localStorage and TanStack Query cache:

```
localStorage.removeItem("access_token");
queryClient.clear();
navigate("/login");
```

---

13. Gamification UI Feedback

13.1 Why Gamification Feedback Matters

Gamification only feels rewarding if the user sees immediate feedback. FinQuest provides three types of feedback:

1. **XP Float Animation**: A floating "+10 XP" appears when adding a transaction
2. **Achievement Toast**: A notification appears when unlocking a badge
3. **Level-Up Modal**: A full-screen celebration when reaching a new level

13.2 GamificationContext

`GamificationContext` manages this UI state:

```
interface GamificationState {
  xpGained: number | null;
  achievementsUnlocked: Achievement[];
  levelUp: boolean;
}
```

13.3 How It Connects to the Backend

When the backend returns a `GamificationDelta` after creating a transaction, the frontend triggers the appropriate feedback:

```
const delta = response.data.gamification_delta;
if (delta.xp_gained > 0) triggerXpGain(delta.xp_gained, "transaction");
if (delta.level_up) triggerLevelUp();
delta.achievements_unlocked.forEach(triggerAchievement);
```

---

14. Data Visualization: Recharts

14.1 What Is Recharts?

Recharts is a composable charting library built with React and D3.js.

14.2 Why Recharts?

Reason 1: React-Native API

Recharts components are React components:

```
<PieChart>
  <Pie data={categoryData} dataKey="amount" nameKey="name" />
  <Tooltip />
  <Legend />
</PieChart>
```

Reason 2: Responsive Charts

Recharts charts automatically resize to their container, which is essential for mobile layouts.

Reason 3: Customizable

Colors, tooltips, legends, and animations can all be customized to match FinQuest's theme.

Charts Used in FinQuest

- **Pie Chart**: Spending breakdown by category
- **Line Chart**: Monthly income vs expense trend
- **Progress Bars**: Budget usage and goal completion

---

15. Forms: React Hook Form + Zod

15.1 What Is React Hook Form?

React Hook Form is a performant form library that minimizes re-renders by using uncontrolled components with refs.

15.2 Why React Hook Form?

Reason 1: Performance

Unlike form libraries that trigger a re-render on every keystroke, React Hook Form uses refs for better performance.

Reason 2: Simple API

Forms are easy to write and maintain:

```
const { register, handleSubmit, formState: { errors } } = useForm();
```

15.3 What Is Zod?

Zod is a TypeScript-first schema validation library.

15.4 Why Zod?

Zod schemas define exactly what a form should contain:

```
const loginSchema = z.object({
  username: z.string().min(3),
  password: z.string().min(8),
});
```

Combined with React Hook Form's resolver, this provides:
- Client-side validation before submission
- TypeScript types from the schema
- Clear error messages

---

16. API Integration

16.1 Two-Tier API Architecture

Tier 1: Low-Level Client (`api/client.ts`)
- Wraps `fetch()`
- Adds `Authorization` header from `localStorage`
- Parses `ApiResponse<T>` envelopes
- Throws on HTTP errors

Tier 2: Domain Services (`services/api.ts`)
- Typed functions for each API domain
- Example:

```
export const transactionApi = {
  create: (data: TransactionCreateInput) ⇒
    api.post("/transactions", data),
  list: (params?: TransactionListParams) ⇒
    api.get(`/transactions?${new URLSearchParams(params)}`),
};
```

16.2 Why This Architecture?

- **DRY**: Common logic (headers, error handling, parsing) lives in one place
- **Type Safety**: Each service method has explicit input/output types
- **Testability**: The client can be mocked for tests

---

17. Component Architecture

17.1 Layout Component

`Layout.tsx` is the application shell:
- Sidebar navigation
- Theme toggle
- Gamification widget (level, XP, streak)
- Achievement toast notifications
- Level-up modal overlay

17.2 Page Components

Pages are data containers:
- Fetch data with TanStack Query
- Handle mutations
- Pass data to presentational components

17.3 UI Components

The `components/ui/` directory contains reusable primitives from shadcn/ui:
- Button, Card, Input, Dialog
- Select, Tabs, Table, Switch
- Calendar, Chart, Carousel

17.4 Why This Separation?

- **Reusability**: UI components are used across multiple pages
- **Testability**: Components can be tested in isolation
- **Maintainability**: Changes to layout don't affect business logic

---

18. Performance Considerations

18.1 Current Optimizations

Technique	Implementation
Caching	TanStack Query caches server data
Lazy Loading	Pages loaded on demand via React Router
CSS Variables	Theme switching without re-renders
Tailwind Purge	Only used styles in production bundle

18.2 Known Limitations

Issue	Cause	Solution
Large JS bundle (~950KB)	Many shadcn/ui components and Recharts	Code splitting with dynamic imports
No service worker	Offline access not implemented	Add PWA support
All charts load upfront	Dashboard includes all charts	Lazy load below-the-fold charts

---

19. Future Frontend Roadmap

Feature	Description
Code Splitting	Reduce initial bundle size with `React.lazy()`
PWA Support	Service worker for offline access and installability
Mobile App	React Native companion app
Real-time Updates	WebSocket or Server-Sent Events for notifications
Advanced Animations	Framer Motion for smoother transitions
Virtualized Lists	React Window for large transaction histories

---

20. Conclusion

FinQuest's frontend is built with a modern, type-safe React stack. React 19 provides the component foundation, Vite enables fast development, TypeScript prevents bugs, Tailwind accelerates styling, TanStack Query handles server state, and shadcn/ui provides accessible components.

Every choice was made to balance development speed, code quality, and user experience. The architecture is clean enough for a final-year project and extensible enough for future enhancements.

Document generated: May 2026
FinQuest v1.0.0 — Frontend Architecture & Technology Choices